

Implementação Paralela de Random Forest usando OpenMP

Eduardo Birkheuer da Silva

João Pedro Ferreira Pereira

Walter Frank

6 de outubro de 2025



INF01017 - Programação Distribuída e Paralela

Sumário

1	Introdução	3
1.1	Visão Geral do Algoritmo Random Forest	3
1.2	Objetivos	3
2	Detalhes da Implementação	3
2.1	Implementação Sequencial	3
2.2	Estratégia de Implementação Paralela	4
2.2.1	Abordagem de Paralelização	4
2.2.2	Decisões de Design e Justificativas	5
3	Configuração Experimental	6
3.1	Ambiente de Hardware e Software	6
3.2	Conjuntos de Dados	6
3.3	Metodologia de Teste	6
4	Resultados de Desempenho	7
4.1	Análise do Tempo de Execução	7
4.2	Análise de Speedup	7
4.3	Análise de Eficiência	8
5	Análise do Intel VTune Profiler	9
5.1	Análise de Snapshot de Desempenho	9
5.2	Análise de Hotspots	9
6	Conclusões	10

1 Introdução

Este relatório apresenta a implementação e análise de um algoritmo Random Forest paralelo usando OpenMP. O trabalho visa demonstrar melhorias de desempenho através da paralelização e analisar o comportamento da aplicação usando o Intel VTune Profiler.

1.1 Visão Geral do Algoritmo Random Forest

Floresta aleatória ou floresta de decisão é um método de aprendizado de máquina do tipo ensemble que constrói múltiplas árvores de decisão durante o treinamento para realizar a classificação de dados ou regressão. No caso da classificação, a previsão final é dada por votação majoritária simples das árvores. Já na regressão, a previsão final é dada pela média das previsões das árvores.

1.2 Objetivos

- Implementar versões sequencial e paralela do Random Forest
- Analisar melhorias de desempenho com diferentes números de threads
- Identificar gargalos e oportunidades de otimização usando VTune
- Avaliar escalabilidade e eficiência da implementação paralela.

2 Detalhes da Implementação

2.1 Implementação Sequencial

A implementação sequencial do Random Forest segue o algoritmo clássico proposto por Breiman, combinando técnicas de bagging (bootstrap aggregating) com aleatoriedade de características para criar um ensemble de árvores de decisão.

Estrutura de Dados Principal:

O sistema utiliza uma estrutura modular composta por:

- **Dataset:** Estrutura para armazenar características (features) e rótulos (labels) dos dados
- **TreeNode:** Nó de árvore contendo índice da característica, threshold de divisão, filhos e predição (para folhas)
- **DecisionTree:** Árvore de decisão implementada como array de nós para eficiência de memória
- **RandomForest:** Ensemble contendo array de árvores e hiperparâmetros

Algoritmo de Treinamento:

O processo de treinamento segue os seguintes passos:

1. **Amostragem Bootstrap:** Para cada árvore, é criada uma amostra bootstrap do conjunto de treinamento original, mantendo o mesmo tamanho mas permitindo repetições de amostras.

2. **Seleção Aleatória de Características:** Cada árvore utiliza apenas um subconjunto aleatório de características, calculado como $\sqrt{n_{features}}$ por padrão, selecionadas sem reposição.

3. **Construção da Árvore de Decisão:** Cada árvore é construída recursivamente usando um algoritmo guloso similar ao CART:

- **Critério de Divisão:** Impureza de Gini ($Gini = 1 - \sum_i p_i^2$)
- **Busca Exaustiva:** Testa todas as características disponíveis e todos os thresholds possíveis

- **Estratégia de Threshold:** Utiliza pontos médios entre valores consecutivos ordenados
- **Divisão Binária:** Cada nó interno divide amostras em dois grupos: $feature \leq threshold$ (esquerda) e $feature > threshold$ (direita)

4. **CrITÉrios de Parada:** A construção de cada árvore para quando:

- Profundidade máxima atingida (padrão: 10 níveis)
- Número mínimo de amostras para divisão não alcançado (padrão: 2 amostras)
- Nó completamente puro (impureza de Gini = 0)
- Nenhuma divisão melhora a impureza

Algoritmo de Predição:

A predição segue um processo de votação majoritária:

1. Cada árvore faz uma predição individual navegando desde a raiz até uma folha
2. A navegação segue as condições de divisão: se $feature[i] \leq threshold$, vai para filho esquerdo, senão para o direito
3. Nas folhas, retorna a classe majoritária das amostras de treinamento que chegaram àquele nó
4. A predição final do ensemble é a classe mais votada entre todas as árvores

Componentes Principais:

- Construção de árvore de decisão usando algoritmo guloso similar a CART
- Amostragem bootstrap do conjunto de treinamento para cada árvore
- Seleção aleatória de subconjunto de características (features) para cada árvore
- Divisão recursiva de nodos buscando thresholds e features que minimizam a impureza de Gini
- Critérios de parada: profundidade máxima ou número mínimo de amostras por nodo
- Agregação de predições através de votação majoritária (classificação) ou média (regressão)

2.2 Estratégia de Implementação Paralela

2.2.1 Abordagem de Paralelização

A paralelização do algoritmo Random Forest foi implementada seguindo uma estratégia de múltiplos níveis, aproveitando as características naturalmente paralelas do algoritmo. A abordagem principal baseia-se no fato de que o treinamento de cada árvore da floresta é completamente independente, permitindo paralelização em nível de árvore. Adicionalmente, foram identificados e paralelizados outros pontos críticos do algoritmo para maximizar o desempenho.

A estratégia de paralelização implementada contempla quatro principais áreas de otimização:

1. **Paralelização do Treinamento de Árvores:** Distribuição das árvores da floresta entre threads diferentes
2. **Paralelização da Busca por Melhor Split:** Distribuição dos potenciais pontos de divisão entre threads

3. **Paralelização da Predição:** Distribuição das árvores durante o processo de inferência
4. **Paralelização da Avaliação de Precisão:** Processamento paralelo das amostras de teste

Diretivas OpenMP Utilizadas:

- `#pragma omp parallel for schedule(static)` - Distribuição estática de iterações entre threads para treinamento e predição
- `#pragma omp critical` - Proteção de seções críticas durante atualizações de variáveis compartilhadas
- `#pragma omp reduction(+:correct_predictions)` - Redução paralela para soma thread-safe na avaliação de precisão
- `#pragma omp atomic capture` - Operações atômicas para contadores compartilhados
- `#pragma omp parallel` com `#pragma omp for nowait` - Regiões paralelas com distribuição de trabalho sem sincronização

2.2.2 Decisões de Design e Justificativas

Decisão 1: Paralelização do Treinamento de Árvores da Floresta

- **Justificativa:** Esta foi a primeira e mais óbvia otimização, visto que o treinamento de cada árvore é totalmente independente das demais. Cada árvore utiliza uma amostra bootstrap diferente e um subconjunto aleatório de características, não havendo dependências entre as árvores.
- **Implementação:** Utilização de `#pragma omp parallel for schedule(static)` na função `train_random_forest`, permitindo que cada thread processe um subconjunto das árvores de forma balanceada.
- **Trade-offs:** Proporciona excelente escalabilidade com overhead mínimo, mas requer cuidado com o gerenciamento de memória para evitar condições de corrida durante a alocação de estruturas das árvores.

Decisão 2: Paralelização da Busca pelo Melhor Split

- **Justificativa:** A busca pelo melhor ponto de divisão representa uma porção significativa do tempo computacional, especialmente com muitas características. Cada potencial split pode ser avaliado independentemente.
- **Implementação:** Implementação de região paralela com `#pragma omp parallel` seguido de `#pragma omp for nowait schedule(static)` na função `find_best_split`, onde cada thread mantém variáveis locais para o melhor resultado e utiliza seção crítica para atualizar o resultado global.
- **Trade-offs:** Melhora significativa de desempenho em datasets com muitas características, mas pode introduzir overhead em datasets pequenos devido ao custo de sincronização.

Decisão 3: Paralelização da Predição das Árvores

- **Justificativa:** Durante a fase de predição, cada árvore pode fazer sua predição independentemente, e a votação majoritária pode ser calculada de forma paralela.

- **Implementação:** Distribuição das árvores entre threads usando `#pragma omp parallel for schedule(static)` na função `predict_random_forest`, permitindo que cada thread processe um subconjunto das árvores da floresta.
- **Trade-offs:** Acelera significativamente a fase de inferência, especialmente importante para florestas grandes, com overhead mínimo para sincronização de resultados.

Decisão 4: Paralelização da Avaliação de Precisão

- **Justificativa:** A avaliação da precisão envolve processar múltiplas amostras de teste independentemente, sendo naturalmente paralelizável.
- **Implementação:** Utilização de `#pragma omp parallel for reduction(+:correct_predictions)` na função `evaluate_accuracy`, permitindo que cada thread processe um subconjunto das amostras e contribua para o contador total de predições corretas de forma thread-safe.
- **Trade-offs:** Oferece speedup proporcional ao número de amostras de teste, com a vantagem de usar redução automática do OpenMP para evitar condições de corrida.

3 Configuração Experimental

3.1 Ambiente de Hardware e Software

- **Processador:** 2 x Intel(R) Xeon(R) E5-2650 v3, 2.30 GHz, 40 threads, 20 cores
- **Memória:** 128 GB DDR4
- **Sistema Operacional:** Debian 12.1
- **Compilador:** Gcc 12.2.0 -Wall -Wextra -O3 -fopenmp -std=c++17
- **Versão OpenMP:** 5.1

3.2 Conjuntos de Dados

Dataset	Tamanho	Características	Amostras
Iris Test	627B	5	30
Student Performance Small	813KB	6	30000
Credit Card Small	4.0MB	31	8000
Company Bankruptcy	11MB	96	6819

3.3 Metodologia de Teste

- **Números de threads testados:** 1, 2, 4, 8, 12, 16, 20, 24
- **Iterações por teste:** 5 (para significância estatística)
- **Métricas coletadas:** Tempo de execução

4 Resultados de Desempenho

4.1 Análise do Tempo de Execução

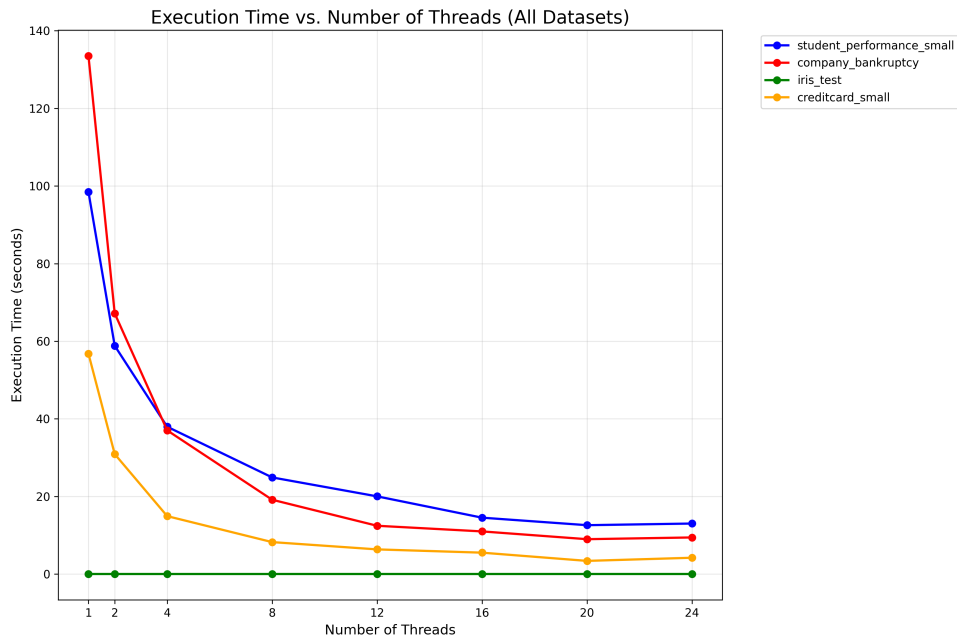


Figura 1: Tempo de Execução para todos os datasets

Em datasets maiores, o programa paralelo reduziu monotonicamente o tempo de execução com o aumento de número de threads até o número de núcleos físicos do sistema, a partir do qual houve uma inflexão com leve aumento de tempo de execução. Em datasets menores, o tempo de execução aumentou com o aumento do número de threads, devido ao overhead de gerenciamento de threads não compensando o ganho de desempenho. O número ótimo de threads é 20, fato confirmado pela especificação da máquina hype5 de 20 núcleos físicos.

4.2 Análise de Speedup

Fórmula de Speedup: $S(p) = T(1)/T(p)$

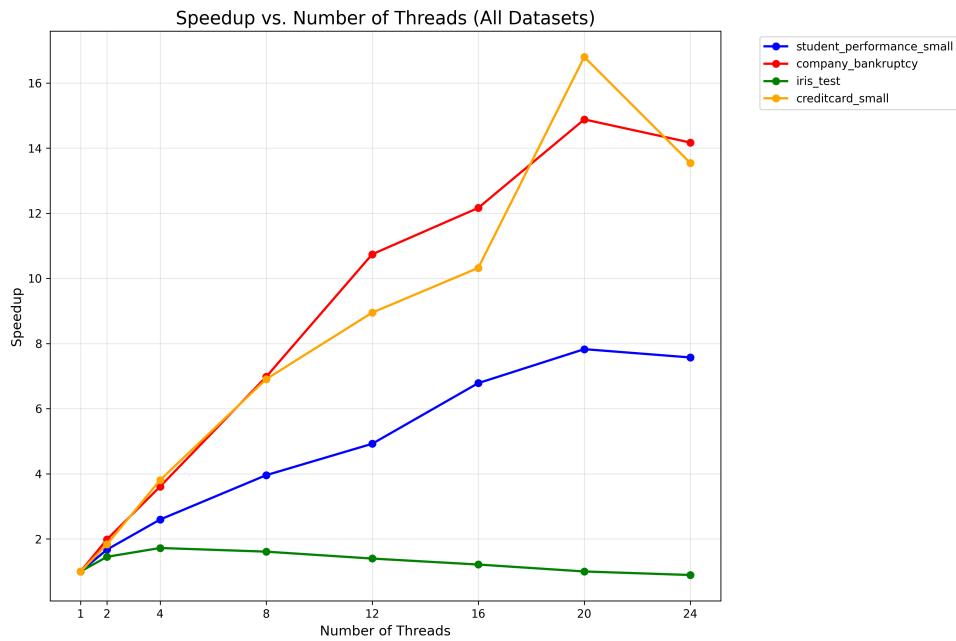


Figura 2: Tempo de Execução para todos os datasets

O gráfico de speedup mostra um aumento de velocidade proporcional ao número de threads para datasets grandes, decrescendo a partir de 20 threads. No dataset iris-test, houve um pequeno speedup até 4 threads, seguida de uma degradação gradual de desempenho.

4.3 Análise de Eficiência

Fórmula de Eficiência: $E(p) = S(p)/p$

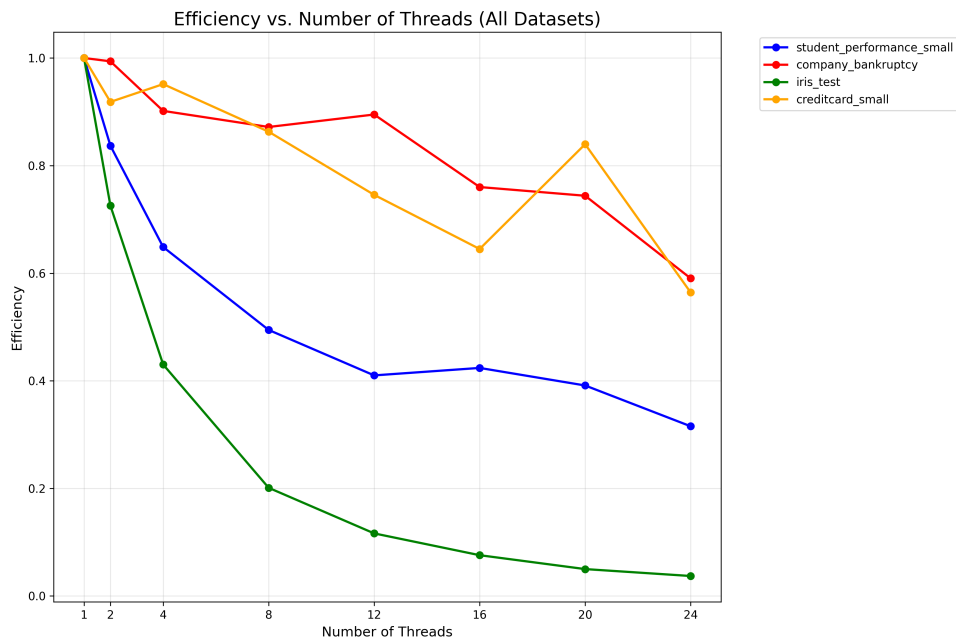


Figura 3: Tempo de Execução para todos os datasets

O gráfico da eficiência destaca grande diferença entre a performance do programa com diferentes datasets. Os datasets creditcard-small e company-bankruptcy obtiveram bons speedups (maior que 0.6) com o aumento do número de threads. O dataset student-performance obteve um speedup menor devido ao baixo número de features e alto número de instâncias, causando um garaglo na função find-best-split, que possui uma variável compartilhada.

5 Análise do Intel VTune Profiler

5.1 Análise de Snapshot de Desempenho

Tabela 1: Performance Analysis Summary - Company Bankruptcy Dataset

Metric	Value
Elapsed Time	33.69s
IPC	0.976
Logical Core Utilization	40.0%
Physical Core Utilization	80.1%
Microarchitecture Usage	25.5%
Pipeline Bottlenecks	
Retiring	25.5%
Front-End Bound	1.7%
Back-End Bound	46.3%
Bad Speculation	26.6%
Memory Bound	24.1%

Os resultados mostram boa utilização de núcleos físicos (80.1%) mas baixa utilização total de threads (40%), o que pode indicar uma dependência de dados e/ou desbaçamento de carga de trabalho, no caso em que uma árvore se torna muito profunda durante o processo de treinamento, tendo como causa a utilização de amostras bootstrap e randomização de features.

5.2 Análise de Hotspots

Principais Consumidores de CPU:

Tabela 2: Top Hotspots

Function	Module	CPU Time	% of CPU Time
calculate_gini_impurity.part.0	rf_parallel	55.158s	48.1%
find_best_split_omp_fn.0	rf_parallel	51.719s	45.1%
__GI___libc_free	libc.so.6	1.941s	1.7%
__GI___libc_malloc	libc.so.6	1.710s	1.5%
merge	rf_parallel	1.599s	1.4%
[Others]	N/A*	2.492s	2.2%

Histograma de utilização de CPU

Tabela 3: Effective CPU Utilization Histogram

Simultaneously Utilized Logical CPUs	Elapsed Time	Utilization threshold
0	0.0270199255	Idle
1	1.2143333954	Poor
2	0.0761810943	Poor
3	0.2980148229	Poor
4	0.2852723783	Poor
5	0.3720010126	Poor
6	0.1807865672	Poor
7	1.1572130455	Ok
8	12.5599649059	Ideal

A análise de hotspots evidencia que a maior parte da carga de trabalho ocorre no cálculo da impureza de gini e na busca pelo melhor split. O histograma mostra pouco tempo ocioso (0.027s), indicando boa ocupação, e que o programa passa a maior parte do tempo (12,55s) em ocupação máxima.

6 Conclusões

Este trabalho demonstrou com sucesso a implementação e análise de desempenho de um algoritmo Random Forest paralelo utilizando OpenMP. Para datasets adequados, a implementação atingiu eficiência superior a 60% em relação ao desempenho sequencial, especialmente nos datasets Credit Card Small e Company Bankruptcy, validando a estratégia de paralelização adotada.

A implementação multi-nível de paralelização demonstrou ser eficaz, explorando diferentes níveis de paralelismo inerentes ao algoritmo Random Forest, incluindo o treinamento independente de árvores, busca paralela por melhores splits, predição paralela das árvores e avaliação paralela de precisão.

Este trabalho contribui para o entendimento prático da paralelização de algoritmos utilizando OpenMP, demonstrando que a escolha adequada de estratégias de paralelização pode resultar em ganhos significativos de desempenho, especialmente em contextos de processamento de grandes volumes de dados.

Referências

- [1] Documentação do Performance Snapshot <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2023-2/performance-snapshot.html>
- [2] Decision Tree Classification Clearly Explained! <https://www.youtube.com/watch?v=sgQAhG5Q7iY&t=464s>
- [3] Decision Tree Classification in Python (from scratch!) <https://www.youtube.com/watch?v=sgQAhG5Q7iY&t=464s>
- [4] Random Forest Algorithm Clearly Explained! <https://www.youtube.com/watch?v=v6VJ2R066Ag>